

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_634e841e-51b\agent-ab051a53c2b1a21b8.jsonl`  
- SHA-256: `851883d41e5ccd46da5b19e22ec725f457a2b24aff228f0ef799331a73785552`  
- Source modified: `2026-07-06T08:43:17+00:00`  
- Imported at: `2026-07-08T16:00:06+00:00`  
- project: `wf_634e841e-51b`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T08:41:48.063Z)

Reviewing GitHub PR #38 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-d-command-service -> main, one commit 22abe72).
It implements ADR 0011 Track D: a REFACTOR that extracts command parsing + config-mutation logic from the
Telethon adapter into a provider-independent CommandService. THE PRIMARY REVIEW RISK IS BEHAVIORAL
EQUIVALENCE: the new code must behave identically to the old inline logic for every command.

Changed files ONLY: src/tg_video_filter/command_service.py (NEW),
src/tg_video_filter/commands.py (refactored
407->~120 line thin Telethon adapter), tests/test_command_service.py (NEW, 20 tests). Diff
+650/-376.
tests/test_commands.py is UNCHANGED and still passes (backwards-compat surface).

To compare against the pre-refactor behaviour, read the OLD file:
git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/commands.py
The new logic lives in command_service.py (CommandService.handle + parsers + formatters) and the new
commands.py delegates to it. Key already-checked facts (verify independently, don't just trust):
- regex patterns are byte-identical old vs new; branch order is the same.
- old handler did text.rstrip() then per-branch logic; new handler passes raw text and handle() does text.rstrip().
- old _update_config RELOADED cfg from db each call; new CommandService._update uses the passed current_cfg,
and the new adapter loads current_cfg = db.load_filter_config(...) fresh right before handle(). Assess equivalence.
- both old and new use cfg.model_copy(update=...) which SKIPS pydantic validation (pre-existing, not new).

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
- A STRONG equivalence check: feed a battery of command strings through BOTH the new CommandService.handle() and (on a checkout of main, or by reasoning from the old source) the old handler, and diff reply_text + resulting saved FilterConfig. Report ANY divergence (different reply string, different config mutation, different validation threshold, different branch taken, different order).
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check clean; ruff format clean; pytest
482 passed @ 97.05% (floor 80%, up from 96.52%).
- Report ONLY defects in THIS PR's diff. For a refactor, a BEHAVIORAL DIVERGENCE from the old

code is the

highest-value finding. Also flag: lost functionality, broken re-exports, new bugs in extracted logic.

No praise. Concrete failure scenario per finding. Rank most-severe first. Honest severity.

- Severity: blocker (wrong behaviour users hit / lost command), high (real divergence, narrow trigger),

medium (edge divergence or latent), low (hygiene/style).

ADVERSARIALLY VERIFY this single finding ? try to REFUTE it. Read the exact cited code (and the old code via

git show main:... when it's an equivalence claim), trace real control flow, run a repro with

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful.

CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour (for

equivalence findings: the new behaviour must actually differ from the old). REFUTED if the code matches old

behaviour / handles it / trigger can't occur / it's pre-existing. PLAUSIBLE if unsettled. Give corrected severity.

FINDING (dim tests):

title: test_command_service.py never asserts the persisted config mutation ? a wrong field/value in CommandService._update would pass all 20 service tests

severity: medium

where: tests/test_command_service.py:87

summary: The new service tests only assert result.config_changed is True plus a substring of reply_text; none reload the config from the DB to verify the correct FilterConfig field was actually mutated to the correct value.

failure_scenario: Swap the field_map in command_service.py so 'gif' maps to 'exclude_round' (or change _update to write max_size_mb into min_duration_s). test_set_exclude_gif_on still passes because the reply label 'GIF/animated' + 'ON' is derived from labels[what], decoupled from field_map, and config_changed stays True; test_set_size passes because '100MB' still appears in the reply. The persisted-field regression is invisible to test_command_service.py ? it is caught only incidentally by tests/test_commands.py (e.g. loaded.max_size_mb == 512 at line 541), which exercises the same _update via the adapter. The dedicated service test file, the natural home for that assertion, does not verify its core responsibility (correct persistence).

evidence: grep for load_filter_config / .max_size_mb / .exclude_ across

tests/test_command_service.py returns zero hits ? no test loads config back. Contrast

tests/test_commands.py:541 `assert loaded.max\_size\_mb == 512`. Confirmed reply label is

independent of field_map, so a swap is not caught by the service tests.

recommendation: In the success-path service tests (test_set_size, test_set_length, test_set_min_length, test_set_exclude_*, test_set_min_resolution) add an assertion that reloads: `loaded = await db.load\_filter\_config(conn, USER\_ID); assert loaded.max\_size\_mb == 100` etc., so the service file independently verifies the field/value that was persisted.

2. assistant (2026-07-06T08:41:54.087Z)

I'll adversarially verify this finding about test coverage gaps in the new service tests.

3. user (2026-07-06T08:41:54.255Z)

```
1      """Tests for the provider-independent CommandService (command_service.py).
2
3      These tests exercise command parsing and config mutation without any
4      Telethon objects ? pure strings in, CommandResult out. The Telethon
5      adapter (commands.py) is tested separately via test_commands.py.
6      """
7
8      from __future__ import annotations
9
10     import aiosqlite
11     import pytest
12
13     from tg_video_filter import db
14     from tg_video_filter.command_service import (
15         CommandService,
```

```

16         format_commands,
17         parse_duration_s,
18         parse_size_mb,
19     )
20 from tg_video_filter.models import FilterConfig
21
22 USER_ID = 111
23
24
25 @pytest.fixture
26 async def conn() -> aiosqlite.Connection:
27     """An in-memory SQLite connection with a seeded user and default config."""
28     c = await aiosqlite.connect(":memory:")
29     await db._apply_schema(c)
30     await db.upsert_user(
31         c,
32         db.User(id=USER_ID, phone="+15555550100"), # type: ignore[attr-defined]
33     )
34     await db.save_filter_config(c, USER_ID, FilterConfig())
35     yield c
36     await c.close()
37
38
39 @pytest.fixture
40 def svc() -> CommandService:
41     return CommandService()
42
43
44 # --- unit parser tests -----
45
46
47 @pytest.mark.parametrize(
48     "value, unit, expected",
49     [
50         ("10", "MB", 10),
51         ("1", "GB", 1024),
52         ("500", "KB", 1),
53         ("5", "", 5),
54         ("2", "TB", 2 * 1024 * 1024),
55     ],
56 )
57 def test_parse_size_mb(value: str, unit: str, expected: int) -> None:
58     assert parse_size_mb(value, unit) == expected
59
60
61 def test_parse_size_mb_unknown_unit_raises() -> None:
62     with pytest.raises(ValueError):
63         parse_size_mb("10", "XB")
64
65
66 @pytest.mark.parametrize(
67     "value, unit, expected",
68     [
69         ("15", "m", 900),
70         ("2", "h", 7200),
71         ("90", "s", 90),
72         ("5", "", 5),
73     ],
74 )
75 def test_parse_duration_s(value: str, unit: str, expected: int) -> None:
76     assert parse_duration_s(value, unit) == expected
77
78
79 def test_parse_duration_s_unknown_unit_raises() -> None:
80     with pytest.raises(ValueError):

```

```

81         parse_duration_s("10", "x")
82
83
84     # --- /set size -----
85
86
87     async def test_set_size(conn: aiomysqlite.Connection, svc: CommandService) -> None:
88         result = await svc.handle(USER_ID, "/set size 100MB", conn, FilterConfig())
89         assert "100MB" in result.reply_text
90         assert result.config_changed is True
91
92
93     async def test_set_size_invalid_format_returns_empty(
94         conn: aiomysqlite.Connection, svc: CommandService
95     ) -> None:
96         """Unparseable input that doesn't match the regex returns empty result (silent)."""
97         result = await svc.handle(USER_ID, "/set size not-a-number", conn, FilterConfig())
98         assert result.reply_text == "" # regex doesn't match, no reply
99         assert result.config_changed is False
100
101
102     async def test_set_size_below_minimum(conn: aiomysqlite.Connection, svc: CommandService)
-> None:
103         result = await svc.handle(USER_ID, "/set size 0MB", conn, FilterConfig())
104         assert "at least 1 MB" in result.reply_text
105         assert result.config_changed is False
106
107
108     # --- /set length -----
109
110
111     async def test_set_length(conn: aiomysqlite.Connection, svc: CommandService) -> None:
112         result = await svc.handle(USER_ID, "/set length 15m", conn, FilterConfig())
113         assert "15m" in result.reply_text or "900s" in result.reply_text
114         assert result.config_changed is True
115
116
117     async def test_set_length_below_min(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
118         cfg = FilterConfig(min_duration_s=60)
119         result = await svc.handle(USER_ID, "/set length 30s", conn, cfg)
120         assert "must be >= min duration" in result.reply_text
121         assert result.config_changed is False
122
123
124     # --- /set min length -----
125
126
127     async def test_set_min_length(conn: aiomysqlite.Connection, svc: CommandService) -> None:
128         result = await svc.handle(USER_ID, "/set min length 5s", conn, FilterConfig())
129         assert "5s" in result.reply_text
130         assert result.config_changed is True
131
132
133     async def test_set_min_length_above_max(conn: aiomysqlite.Connection, svc: CommandService)
-> None:
134         cfg = FilterConfig(max_duration_s=10)
135         result = await svc.handle(USER_ID, "/set min length 60s", conn, cfg)
136         assert "must be <= max duration" in result.reply_text
137         assert result.config_changed is False
138
139
140     # --- /set min resolution -----
141
142

```

```

143     async def test_set_min_resolution(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
144         result = await svc.handle(USER_ID, "/set min resolution 720p", conn, FilterConfig())
145         assert "720p" in result.reply_text
146         assert result.config_changed is True
147
148
149     async def test_set_min_resolution_below_min(
150         conn: aiomysqlite.Connection, svc: CommandService
151     ) -> None:
152         result = await svc.handle(USER_ID, "/set min resolution 0p", conn, FilterConfig())
153         assert "at least 1p" in result.reply_text
154         assert result.config_changed is False
155
156
157     async def test_set_min_res_help_on_malformed(
158         conn: aiomysqlite.Connection, svc: CommandService
159     ) -> None:
160         result = await svc.handle(USER_ID, "/set min resolution abc", conn, FilterConfig())
161         assert "Usage:" in result.reply_text
162
163
164     # --- /set exclude -----
165
166
167     async def test_set_exclude_gif_on(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
168         result = await svc.handle(USER_ID, "/set exclude gif on", conn, FilterConfig())
169         assert "ON" in result.reply_text
170         assert result.config_changed is True
171
172
173     async def test_set_exclude_round_off(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
174         result = await svc.handle(USER_ID, "/set exclude round off", conn, FilterConfig())
175         assert "OFF" in result.reply_text
176         assert result.config_changed is True
177
178
179     async def test_set_exclude_help_on_malformed(
180         conn: aiomysqlite.Connection, svc: CommandService
181     ) -> None:
182         result = await svc.handle(USER_ID, "/set exclude gif true", conn, FilterConfig())
183         assert "Usage:" in result.reply_text
184         assert result.config_changed is False
185
186
187     # --- /show config -----
188
189
190     async def test_show_config(conn: aiomysqlite.Connection, svc: CommandService) -> None:
191         result = await svc.handle(USER_ID, "/show config", conn, FilterConfig())
192         assert "Current Filter Configuration" in result.reply_text
193         assert result.config_changed is False
194
195
196     # --- /show commands -----
197
198
199     async def test_show_commands(conn: aiomysqlite.Connection, svc: CommandService) -> None:
200         result = await svc.handle(USER_ID, "/show commands", conn, FilterConfig())
201         assert "Available Commands" in result.reply_text
202         assert result.config_changed is False
203
204

```

```

205     # --- unrecognized command -----
206
207
208     async def test_unrecognized_returns_empty(conn: aiomysqlite.Connection, svc:
CommandService) -> None:
209         result = await svc.handle(USER_ID, "hello world", conn, FilterConfig())
210         assert result.reply_text == ""
211         assert result.config_changed is False
212
213
214     # --- format_commands -----
215
216
217     def test_format_commands_includes_all_sections() -> None:
218         text = format_commands()
219         assert "/show config" in text
220         assert "/set size" in text
221         assert "/set exclude" in text
222         assert "/backfill" in text
223

```

4. assistant (2026-07-06T08:41:58.444Z)

The finding claims the service tests never assert persisted config mutation. The test file confirms this. Now let me verify the two key decoupling claims: (1) the reply label comes from a `labels` map independent of `field\_map`, and (2) `test\_set\_size`'s "100MB" reply is independent of which field gets persisted. Let me read the service code.

5. user (2026-07-06T08:41:58.810Z)

```

1     """Provider-independent command parsing and execution (ADR 0011, Track D).
2
3     Extracts command logic from the Telethon adapter (`commands.py`) so the
4     same ``/set``, ``/show`` commands work identically through a Bot API
5     front-door. The service is pure: strings in, ``CommandResult`` out ? no
6     Telethon objects, no network calls.
7
8     See ``docs/adr/0007-inline-config-commands.md`` for the command model and
9     ``docs/adr/0008-bot-api-onboarding-front-door.md`` for the Bot API motivation.
10    """
11
12    from __future__ import annotations
13
14    import logging
15    import re
16    from dataclasses import dataclass
17
18    import aiomysqlite
19
20    from tg_video_filter import db
21    from tg_video_filter.models import FilterConfig
22
23    log = logging.getLogger(__name__)
24
25    # Key into the shared engine_holder dict (imported by commands.py's adapter).
26    ENGINE_KEY = "engine"
27
28    # --- regex patterns -----
29
30    _SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmgT]?b?)$", re.IGNORECASE)
31    _LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
32    _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
33    _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$",
re.IGNORECASE)
34    _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)

```

```

35     _SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
36     _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$",
re.IGNORECASE)
37     _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
38     _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
39
40
41     # --- unit parsing -----
42
43
44     def parse_size_mb(value_str: str, unit: str) -> int:
45         """Parse a size value + unit string into mebibytes."""
46         value = int(value_str)
47         unit = unit.lower().rstrip("b")
48         multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
49         if unit in multipliers:
50             raw = value * multipliers[unit]
51             if unit == "k":
52                 raw = max(1, raw // 1024)
53             return raw
54         raise ValueError(f"Unknown size unit: {unit}")
55
56
57     def parse_duration_s(value_str: str, unit: str) -> int:
58         """Parse a duration value + unit string into seconds."""
59         value = int(value_str)
60         unit = unit.lower()
61         multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
62         if unit in multipliers:
63             return value * multipliers[unit]
64         raise ValueError(f"Unknown time unit: {unit}")
65
66
67     # --- formatting helpers -----
68
69
70     def _format_duration(seconds: int | None) -> str:
71         if seconds is None:
72             return "unlimited"
73         if seconds == 0:
74             return "0s"
75         if seconds < 60:
76             return f"{seconds}s"
77         if seconds < 3600:
78             m, s = divmod(seconds, 60)
79             return f"{m}m{s}s" if s else f"{m}m"
80         h, rem = divmod(seconds, 3600)
81         m, s = divmod(rem, 60)
82         parts = f"{h}h"
83         if m:
84             parts += f"{m}m"
85         if s:
86             parts += f"{s}s"
87         return parts
88
89
90     def _format_size(mb: int | None) -> str:
91         if mb is None:
92             return "unlimited"
93         if mb == 0:
94             return "0MB"
95         if mb >= 1024:
96             gb = mb / 1024
97             if gb == int(gb):
98                 return f"{int(gb)}GB"

```

```

99         return f"{gb:.1f}GB"
100     return f"{mb}MB"
101
102
103     def format_config(cfg: FilterConfig) -> str:
104         """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
105         lines = [
106             "? **Current Filter Configuration**",
107             "",
108             f"? **Duration:** {_format_duration(cfg.min_duration_s)} - {_format_duration(cfg.max_duration_s)}",
109             f"? **Min resolution:** {' ' + str(cfg.target_height) + 'p' if cfg.target_height else 'disabled'}",
110             f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
111             f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
112             f"? **Delivery target:** `{cfg.delivery_target}`",
113             "",
114             "? **Content exclusions:**",
115             f"    GIF/animated: {' excluded' if cfg.exclude_gif else '? allowed'}",
116             f"    Round videos: {' excluded' if cfg.exclude_round else '? allowed'}",
117             f"    Silent (no audio): {' excluded' if cfg.exclude_nosound else '? allowed'}",
118         ]
119         return "\n".join(lines)
120
121
122     def format_commands() -> str:
123         """Return a help message listing all available commands."""
124         return "\n".join(
125             [
126                 "? **Available Commands**",
127                 "",
128                 "? **Filter config**",
129                 "`/show config` ? Show current filter settings",
130                 "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
131                 "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing `B`",
132                 "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
133                 "    or `/set max length <N><unit>` ? same as above",
134                 "    Units: `s` (default), `m`, `h` ? case-insensitive",
135                 "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length 5s`)",
136                 "    Units: `s` (default), `m`, `h` ? case-insensitive",
137                 "`/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution 720p`)",
138                 "    or `/set min res <N>p` ? same as above",
139                 "",
140                 "? **Content exclusions** (all off by default)",
141                 "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
142                 "`/set exclude round on|off` ? Exclude circular video messages",
143                 "`/set exclude nosound on|off` ? Exclude videos without audio\n",
144                 "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs",
145                 "",
146                 "? **Backfill**",
147                 "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history (default 90d)",
148                 "`/stop` ? Cancel running backfill",
149                 "",
150                 "\u2139ufe0f **Help**",
151                 "`/show commands` ? Show this help message",
152             ]
153         )
154
155
156     # --- command result -----

```



```

157
158
159 @dataclass
160 class CommandResult:
161     """The output of executing a command ? provider-independent.
162
163     The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
164     and ``config_changed`` to decide whether to hot-reload the filter engine.
165     """
166
167     reply_text: str = ""
168     config_changed: bool = False
169
170
171 # --- command service -----
172
173
174 class CommandService:
175     """Provider-independent command execution.
176
177     Parses a command string against an account's current config and DB,
178     returning a ``CommandResult``. No Telethon objects involved ? pure string
179     in, effects out. The Telethon adapter (``commands.py``) handles the
180     transport (checking message.out, sending the reply); the Bot API adapter
181     (future) calls ``handle()`` with the ``from.id`` from the update.
182
183     Usage::
184
185         svc = CommandService()
186         result = await svc.handle(user_id, text, conn, current_cfg)
187         if result.config_changed:
188             engine_holder["engine"] = FilterEngine(current_cfg)
189         await send_reply(result.reply_text)
190     """
191
192 # -- public API -----
193
194 async def handle(
195     self,
196     user_id: int,
197     text: str,
198     conn: aiosqlite.Connection,
199     current_cfg: FilterConfig,
200 ) -> CommandResult:
201     """Parse and execute a command text, returning the reply + change flag."""
202     text = text.rstrip()
203
204     # --- /set size <value><unit> -----
205     m = _SIZE_RE.match(text)
206     if m:
207         value_str, unit = m.group(1), m.group(2) or ""
208         try:
209             new_mb = parse_size_mb(value_str, unit)
210         except ValueError:
211             return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
212         if new_mb < 1:
213             return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
214         cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
215         return CommandResult(
216             f"? Max file size set to
**[_format_size(new_mb)]*.\\n\\n[_format_config(cfg)]",
217             config_changed=True,
218         )

```

```

219
220 # --- /set length <value><unit> -----
221 m = _LENGTH_RE.match(text)
222 if m:
223     value_str, unit = m.group(1), m.group(2) or ""
224     try:
225         new_secs = parse_duration_s(value_str, unit)
226     except ValueError:
227         return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
228     min_secs = current_cfg.min_duration_s or 0
229     if new_secs < min_secs:
230         return CommandResult(
231             f"? Max duration ({_format_duration(new_secs)}) must be "
232             f">= min duration ({_format_duration(min_secs)}).")
233         )
234     cfg = await self._update(conn, user_id, current_cfg,
max_duration_s=new_secs)
235     return CommandResult(
236         f"? Max duration set to
**[_format_duration(new_secs)]**.\n\n{format_config(cfg)}",
237         config_changed=True,
238     )
239
240 # --- /set min length <value><unit> -----
241 m = _MIN_LENGTH_RE.match(text)
242 if m:
243     value_str, unit = m.group(1), m.group(2) or ""
244     try:
245         new_min = parse_duration_s(value_str, unit)
246     except ValueError:
247         return CommandResult("? Invalid duration format. Use e.g. `/set min
length 5s`.")
248     max_secs = current_cfg.max_duration_s or 0
249     if max_secs > 0 and new_min > max_secs:
250         return CommandResult(
251             f"? Min duration ({_format_duration(new_min)}) must be "
252             f"<= max duration ({_format_duration(max_secs)}).")
253         )
254     cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
255     return CommandResult(
256         f"? Min duration set to
**[_format_duration(new_min)]**.\n\n{format_config(cfg)}",
257         config_changed=True,
258     )
259
260 # --- /set min resolution <N>p -----
261 m = _MIN_RES_RE.match(text)
262 if m:
263     new_height = int(m.group(1))
264     if new_height < 1:
265         return CommandResult("? Resolution must be at least 1p.")
266     cfg = await self._update(conn, user_id, current_cfg,
target_height=new_height)
267     return CommandResult(
268         f"? Min resolution set to **{new_height}p**.\n\n{format_config(cfg)}",
269         config_changed=True,
270     )
271
272 if _MIN_RES_HELP_RE.match(text):
273     return CommandResult(
274         "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
275     )
276
277 # --- /set exclude <gif|round|nosound> <on|off> -----

```

```

278         m = _EXCLUDE_RE.match(text)
279         if m:
280             what = m.group(1).lower()
281             state = m.group(2).lower()
282             enabled = state == "on"
283             labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}
284             field_map = {
285                 "gif": "exclude_gif",
286                 "round": "exclude_round",
287                 "nosound": "exclude_nosound",
288             }
289             cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
290             return CommandResult(
291                 f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
292                 f"\n\n{format_config(cfg)}",
293                 config_changed=True,
294             )
295
296         if _EXCLUDE_HELP_RE.match(text):
297             return CommandResult(
298                 "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
299                 "Example: `/set exclude gif on`"
300             )
301
302         # --- /show config -----
303         if _SHOW_CONFIG_RE.match(text):
304             cfg = await db.load_filter_config(conn, user_id)
305             return CommandResult(format_config(cfg))
306
307         # --- /show commands -----
308         if _SHOW_COMMANDS_RE.match(text):
309             return CommandResult(format_commands())
310
311         # Not a recognised command ? return empty result (noise).
312         return CommandResult()
313
314         # -- internal -----
315
316     async def _update(
317         self,
318         conn: aiosqlite.Connection,
319         user_id: int,
320         current: FilterConfig,
321         max_size_mb: int | None = None,
322         max_duration_s: int | None = None,
323         min_duration_s: int | None = None,
324         target_height: int | None = None,
325         exclude_gif: bool | None = None,
326         exclude_round: bool | None = None,
327         exclude_nosound: bool | None = None,
328     ) -> FilterConfig:
329         """Apply partial updates to the account's config, persist, and return."""
330         updates: dict[str, object] = {}
331         if max_size_mb is not None:
332             updates["max_size_mb"] = max_size_mb
333         if max_duration_s is not None:
334             updates["max_duration_s"] = max_duration_s
335         if min_duration_s is not None:
336             updates["min_duration_s"] = min_duration_s
337         if target_height is not None:
338             updates["target_height"] = target_height
339         if exclude_gif is not None:
340             updates["exclude_gif"] = exclude_gif

```

```

341         if exclude_round is not None:
342             updates["exclude_round"] = exclude_round
343         if exclude_nosound is not None:
344             updates["exclude_nosound"] = exclude_nosound
345         cfg = current.model_copy(update=updates) if updates else current
346         await db.save_filter_config(conn, user_id, cfg)
347         log.info(
348             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
349             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
350             user_id,
351             cfg.max_size_mb,
352             cfg.min_duration_s,
353             cfg.max_duration_s,
354             cfg.target_height,
355             cfg.exclude_gif,
356             cfg.exclude_round,
357             cfg.exclude_nosound,
358         )
359         return cfg
360
361
362     __all__ = [
363         "ENGINE_KEY",
364         "CommandResult",
365         "CommandService",
366         "_format_duration",
367         "_format_size",
368         "format_config",
369         "format_commands",
370         "parse_duration_s",
371         "parse_size_mb",
372     ]
373

```

6. assistant (2026-07-06T08:42:07.645Z)

Now I can verify the decoupling claims. Note something important: the finding claims the tests only assert `config\_changed is True` plus a substring, and that a swapped field_map would pass. But there's a subtlety I must check: the reply includes `format\_config(cfg)` where `cfg` is the mutated config. Let me trace whether a field-map swap would actually be caught by the reply substring assertions.

Let me examine the finding's specific claims carefully.

****Claim 1: `test\_set\_exclude\_gif\_on` ? swap gif?exclude_round.**** The test asserts `"ON" in result.reply\_text`. The reply is `f"? {labels[what]} exclusion: \*\*ON\*\*...{format\_config(cfg)}"`. With `what="gif"`, `labels["gif"]` = "GIF/animated" regardless of field_map. If field_map["gif"] were "exclude_round", then `cfg.exclude\_round` becomes True. `format\_config` would show "Round videos: excluded" and "GIF/animated: allowed". But the test only checks `"ON" in reply\_text` ? which is still present from the label line. So yes, the swap passes. Let me verify by running a repro.

7. user (2026-07-06T08:42:17.773Z)

```

reply contains ON: True
exclude_gif: True exclude_round: True
reply contains 100MB: True
max_size_mb: 100 min_duration_s: 5

```

8. assistant (2026-07-06T08:42:22.221Z)

Interesting ? note `exclude\_round: True` and `min\_duration\_s: 5` in the loaded config, but that's because I'm reusing the same connection across both handle calls and the seeded config carries over. Not relevant. The key results: reply contains "ON" and "100MB" regardless.

Now let me actually simulate the mutation the finding describes ? swap the field_map so gif?exclude_round, and change _update to write max_size_mb into min_duration_s ? then run the actual service tests to confirm they still pass.

9. user (2026-07-06T08:42:24.243Z)

backed up

10. user (2026-07-06T08:42:28.008Z)

The file C:/Users/avidu/Projects/Telegram/src/tg_video_filter/command_service.py has been updated successfully. (file state is current in your context ? no need to Read it back)

11. user (2026-07-06T08:42:31.121Z)

The file C:/Users/avidu/Projects/Telegram/src/tg_video_filter/command_service.py has been updated successfully. (file state is current in your context ? no need to Read it back)

12. assistant (2026-07-06T08:42:33.756Z)

Now run just the service tests with both mutations in place.

13. user (2026-07-06T08:42:37.183Z)

src\tg_video_filter\backfill.py	49	49	16	0	0%	17-405
src\tg_video_filter\bootstrap.py	23	23	0	0	0%	17-78
src\tg_video_filter\command_service.py	158	14	66	8	90%	72, 74, 84, 86, 92, 94, 99, 210-211, 226-227, 246-247, 344
src\tg_video_filter\commands.py	44	44	10	0	0%	14-126
src\tg_video_filter\config.py	26	5	2	0	75%	62-64, 69, 79
src\tg_video_filter\db.py	307	142	46	11	50%	195-196, 213-215, 292-312, 323, 343, 379-501, 520, 649-652, 683, 686, 788-791, 829, 844, 858, 893, 906, 919, 968-971, 1030-1039, 1085, 1097-1098, 1110-1139, 1146-1153, 1167-1194, 1204-1211, 1223-1229, 1287-1314, 1321-1327, 1345-1368, 1375-1381, 1394-1406, 1411-1412, 1423-1428, 1449-1453, 1460-1467, 1492-1508, 1531-1534, 1545-1582, 1597, 1610, 1643, 1671-1681, 1749
src\tg_video_filter\delivery.py	102	102	48	0	0%	13-283
src\tg_video_filter\filters__init__.py	39	39	18	0	0%	16-78
src\tg_video_filter\filters\base.py	26	26	0	0	0%	7-63
src\tg_video_filter\filters\content_type.py	23	23	12	0	0%	9-44
src\tg_video_filter\filters\duration.py	15	15	6	0	0%	3-28
src\tg_video_filter\filters\resolution.py	13	13	6	0	0%	3-27
src\tg_video_filter\filters\size.py	18	18	6	0	0%	3-34
src\tg_video_filter\manager.py	36	36	8	0	0%	26-204
src\tg_video_filter\models.py	162	7	8	3	93%	364, 366, 377, 416, 424-426
src\tg_video_filter\telethon_client.py	62	62	26	0	0%	16-358

TOTAL	1111	625	278	22	40%	
FAIL Required test coverage of 80.0% not reached. Total coverage: 40.32%						
28 passed in 0.93s						

14. assistant (2026-07-06T08:42:42.064Z)

Confirmed. **All 28 test items in `test\_command\_service.py` passed** despite both mutations: `gif` mapped to `exclude\_round`, and `max\_size\_mb` writing into `min\_duration\_s`. The only failure is the global coverage floor (an artifact of running one file in isolation), not any assertion. The persistence regression is completely invisible to the service test file.

Now let me confirm the finding's companion claim: that `test\_commands.py:541` DOES catch this (i.e., the regression is only caught incidentally by the adapter tests), to validate the finding's characterization. Let me restore first, then check.

15. user (2026-07-06T08:42:45.240Z)

restored

16. assistant (2026-07-06T08:42:49.101Z)

The reminder says the file was modified intentionally ? but that was my restore from backup, which returned it to the original PR content. It's the correct original. Let me verify the restore is clean and confirm test_commands.py:541 exists as the finding claims.

17. user (2026-07-06T08:42:50.905Z)

```
110:         f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
214:         cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
321:         max_size_mb: int | None = None,
331:         if max_size_mb is not None:
332:             updates["max_size_mb"] = max_size_mb
348:             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
351:             cfg.max_size_mb,
---541---
    event.client.send_message.assert_called_once()
    args, _kwargs = event.client.send_message.call_args
    assert "512MB" in args[1]
    assert "?" in args[1]

    # Persisted to DB.
    loaded = await db.load_filter_config(memory_conn, USER_ID)
    assert loaded.max_size_mb == 512

    # Engine hot-reloaded.
    assert engine_holder[ENGINE_KEY].cfg.max_size_mb == 512
```

18. assistant (2026-07-06T08:43:07.280Z)

Restore is clean (line 332 correct: `max\_size\_mb`?`max\_size\_mb`). And `test\_commands.py:541` confirms `assert loaded.max\_size\_mb == 512` ? the adapter test does reload and assert persistence, matching the finding's characterization exactly.

Let me also verify the claim that `test\_set\_exclude\_gif\_on`'s label is truly decoupled: `labels[what]` uses `what` (from regex group), not `field\_map`. Confirmed at command_service.py:283 ? `labels` is keyed by `what` independent of `field\_map` at :284-288. The "ON"/"OFF" text comes from `state`/`enabled`, also independent of field_map. And note `format\_config(cfg)` is appended, but the tests only substring-match "ON" / "100MB" / "720p" etc., which appear in the fixed label/prefix lines regardless of the mutated field ? so a wrong field survives.

Assessment complete.

****Verdict analysis:****

The finding is a *test-quality* finding, not a behavioral-divergence finding. I verified:

1. The service test file contains zero `load\_filter\_config` calls and zero field-value assertions (confirmed reading the whole file).
2. A deliberate field_map swap (gif?exclude_round) AND a `\_update` mis-mapping (max_size_mb?min_duration_s) both pass all 28 test items in `test\_command\_service.py` ? proven by actually applying both mutations and running the file (28 passed, only the isolated-run coverage floor "failed", which is not an assertion).
3. The reply labels are genuinely decoupled from `field\_map` (labels keyed by `what`; ON/OFF from `state`), and the substring assertions match text that is present regardless of the persisted field.
4. The regression is caught only incidentally by `test\_commands.py:541` (`assert loaded.max\_size\_mb == 512`), as the finding states.

The one caveat on severity: this PR is a REFACTOR of existing, separately-tested logic. `\_update`/field_map behavior IS covered end-to-end by the unchanged `test\_commands.py` adapter tests, so a real regression would not ship undetected by the *suite as a whole*. The gap is that

the new dedicated test file doesn't independently verify its core responsibility (correct persistence) ? a genuine weakness in the new tests, but not a latent product bug and not a behavioral divergence. Medium is slightly generous for a refactor where the behavior is still covered elsewhere; low-to-medium is fair. I'll set medium as the finding is accurate and the recommendation is sound, but note it sits at the low end.

19. user (2026-07-06T08:43:17.756Z)

Structured output provided successfully